NU MOA

Secure Software Development

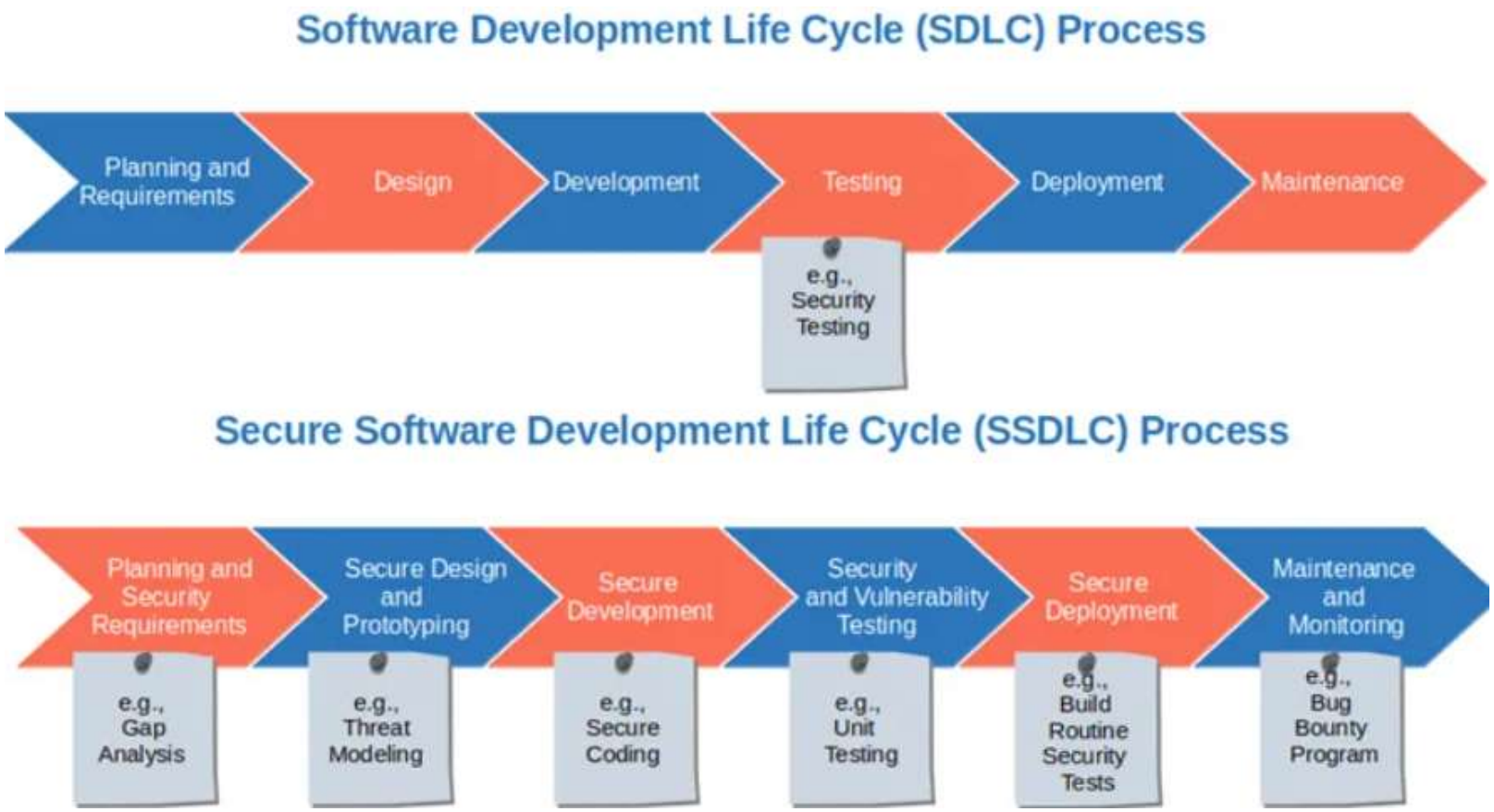
What does Secure Design in Software Development mean?

NU MOA

"Secure-by-design" is an approach to designing and building systems with security in mind. It involves implementing continuous security practices, tools, and controls from the beginning of the software development lifecycle, ensuring that products are inherently secure.

Secure-by-design is rooted in the DevSecOps philosophy – merging development, security, and operations. It helps to foster a culture of security awareness and responsibility across the teams.

Secure Software Development Life Cycle Process



7 Principles of Secure Design in Software Development

1. Security as Code

Security as Code (SaC) transforms security practices from manual processes to automated, scriptable components within the software development lifecycle.

SaC involves writing scripts or predefined code templates to enforce security policies, performing vulnerability assessments, and automating remediation and security scanning processes.

7 Principles of Secure Design in Software Development

2. Secure Defaults

Adopting Secure Defaults is a fundamental principle that mandates systems configurations to be inherently safe.

This approach stems from the understanding that many end-users stick to default settings. By setting these defaults to be highly secure, you can safeguard most users.

Secure Defaults could manifest in robust password policies, multi-factor authentication requirements, and even the default turning off extra services or features that could serve as attack vectors.

7 Principles of Secure Design in Software Development

3. Least Privilege

The Least Privilege principle is a crucial best practice for Identity and Access Management (IAM). And it doesn’t just apply to users - every component in your system (machine identities such as resources and networks) should also operate with the least privilege necessary to perform its function.

Adopting this approach reduces the potential damage of a breach by reducing the accessible attack surface.

7 Principles of Secure Design in Software Development

4. Separation of Duties

Separation of Duties (SoD) is about dividing tasks and privileges to minimize the risk of malicious activity or errors..

No user should have a level of access that allows them to misuse a system on their own.

7 Principles of Secure Design in Software Development

5. Minimize Attack Surface Area

The Attack Surface Area (ASA) represents all the points where an unauthorized user can interact with a system or application. .

Minimizing ASA is about limiting these entry points, thereby shrinking the opportunities for adversaries to exploit vulnerabilities.

7 Principles of Secure Design in Software Development

6. Complete Mediation

Complete mediation mandates that every access request to any system resource is thoroughly authenticated and authorized. This should happen every time, without exception.

7 Principles of Secure Design in Software Development

7. Failing Securely

The principle of failing securely dictates that a system should respond to errors or failures in a way that preserves the overall security of the application.

When something goes wrong, the system should default to a secure state rather than exposing vulnerabilities or sensitive information.

Code Review

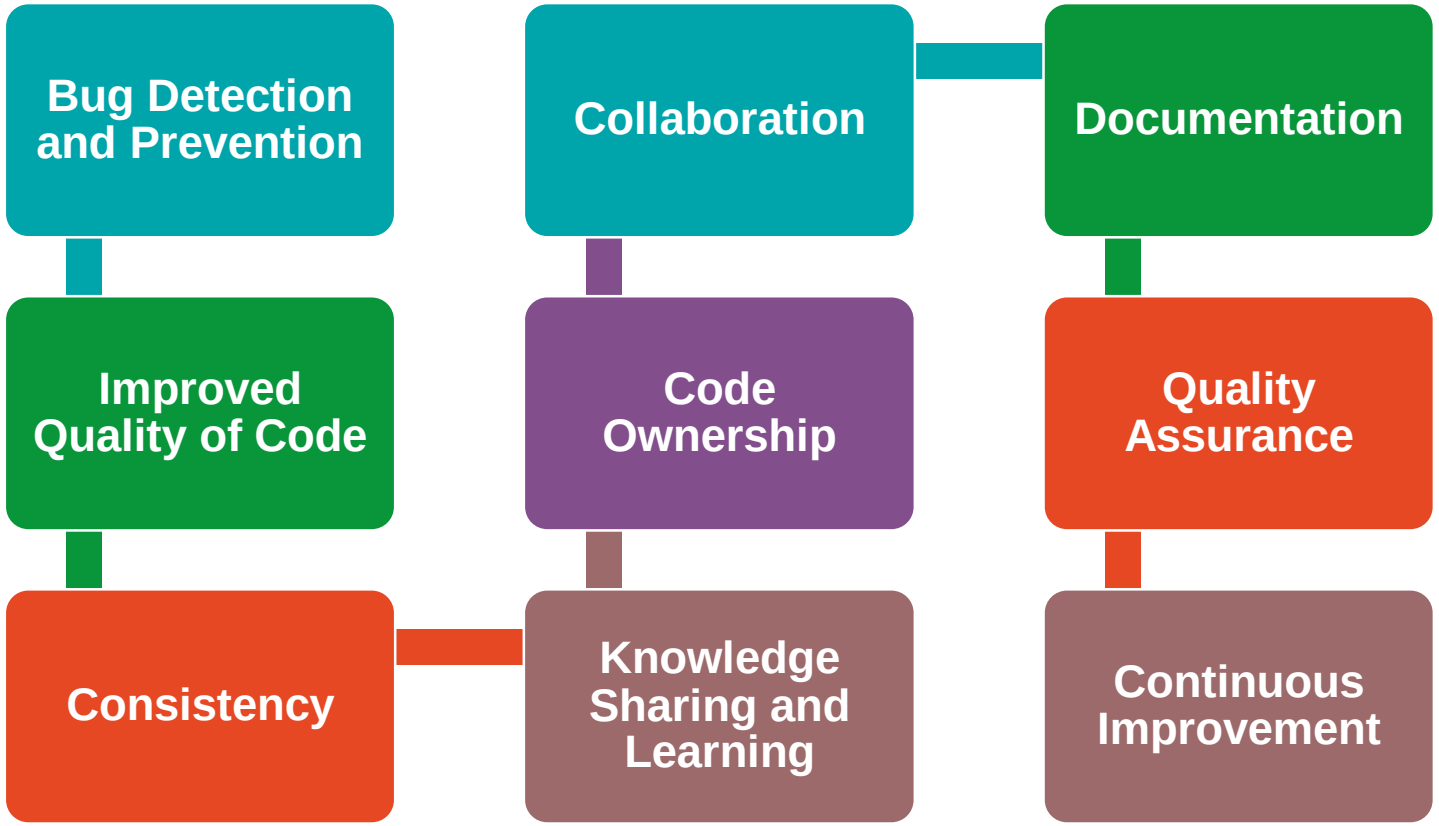
Code Review

Code review is a systematic and collaborative process in software development where one or more developers examine and evaluate another developer’s code to identify issues, provide feedback, and ensure that the code meets quality standards.

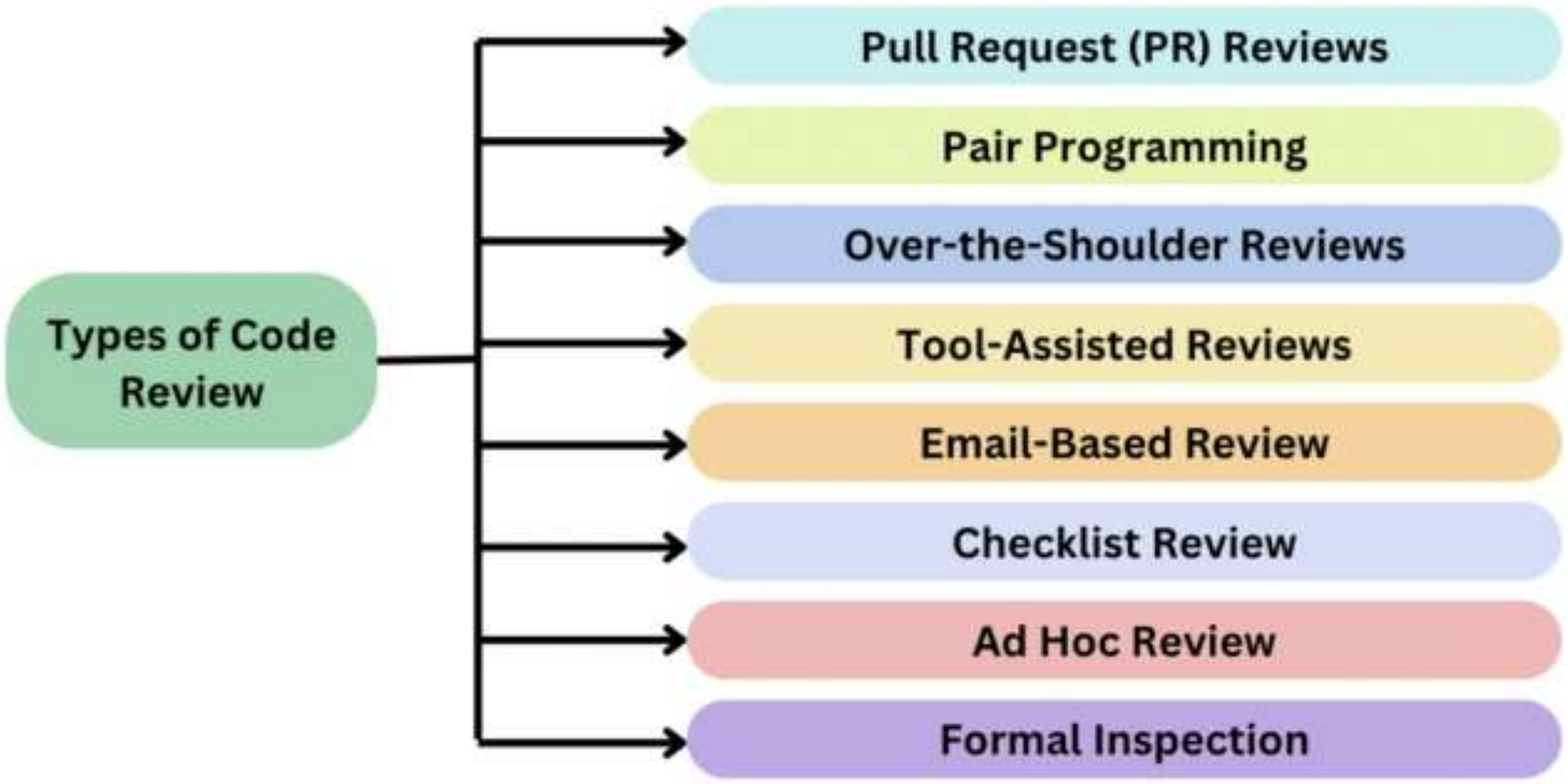
The primary goal is to improve the maintainability, quality, and reliability of the codebase, resulting in the overall success of a project while also promoting knowledge sharing and learning among team members.

NU MOA

Why is Code Review important?



Types of Code Review



Types of Code Review

NU MOA

Pull Request (PR) Reviews: In Git-based version control systems like GitHub, etc., developers often create pull or merge requests to propose changes to the codebase. These pull requests are reviewed by the team members before the changes are merged.

Pair Programming: In pair programming, two developers work together on the same computer, with one writing code while the other reviewing it in real time. This form of code review is highly interactive.

Over-the-Shoulder Reviews: A developer may ask a team member to review their code by physically sitting together and going through the code on the computer screen.

Tool-Assisted Reviews: Various tools and platforms are available to facilitate code reviews, such as GitHub, GitLab, Bitbucket, and code review-specific tools like Crucible and Review Board.

Types of Code Review

NU MOA

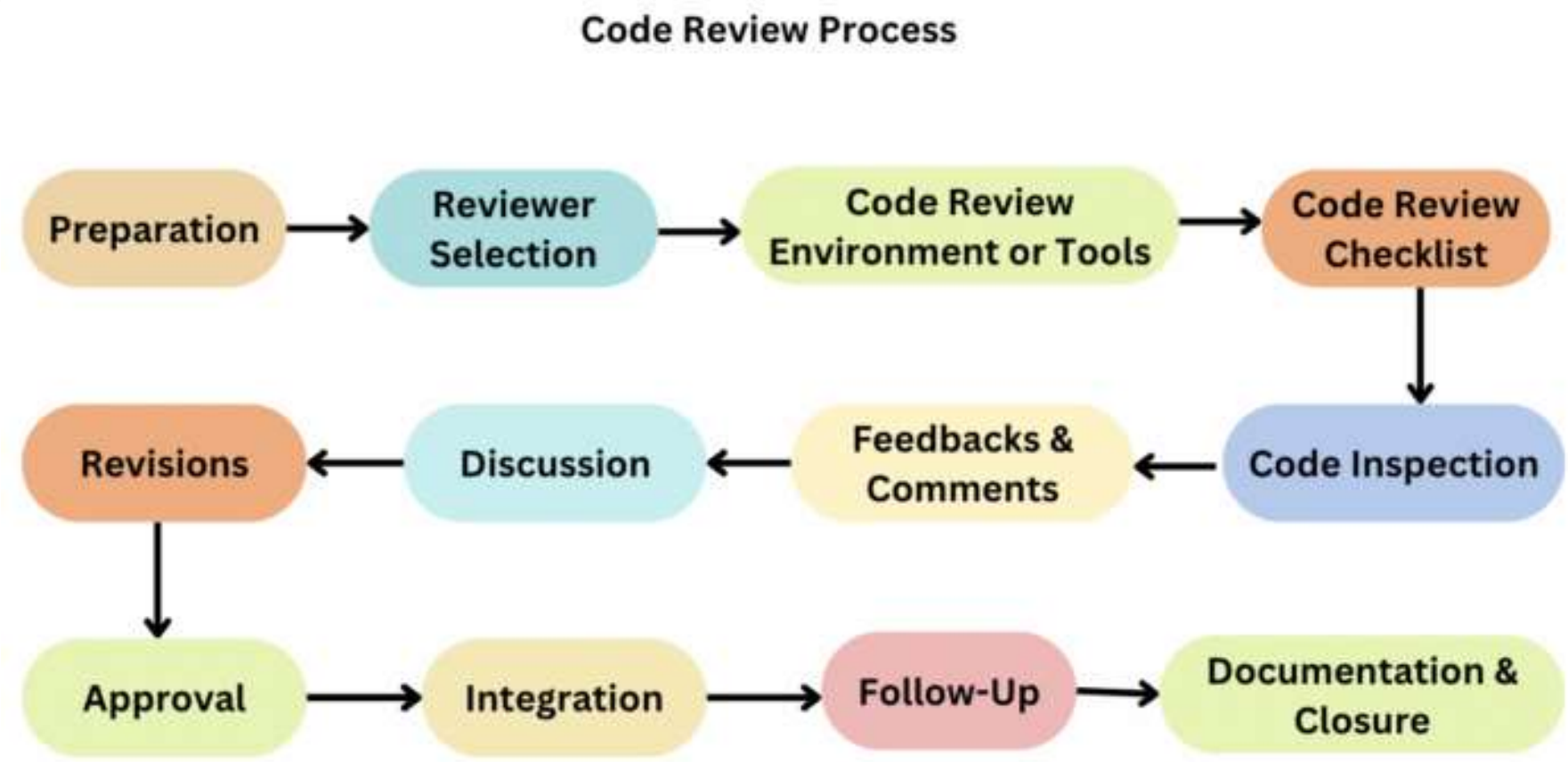
Email-Based Review: In email-based reviews, code changes are sent via email, and the reviewers provide feedback and comments in response. The discussion occurs over email threads.

Checklist Review: In a checklist review, a checklist is used to evaluate the code. Reviewers go through the list and check off items as they are reviewed.

Ad Hoc Review: Ad hoc reviews are informal and spontaneous. Developers may spontaneously ask team members to take a quick look at their code or discuss changes without a formal process.

Formal Inspection: Formal inspections are a structured form of code review that follows a predefined process. They often involve a dedicated inspection team and detailed documentation.

Code Review Process



Code Review Process

NU MOA

1. Preparation: The developer who has completed a piece of code or feature initiates the code review by creating a review request or notifying the team.

2. Reviewer Selection: One or more team members are assigned as reviewers. Reviewers must have the relevant expertise to assess the code properly.

3. Code Review Environment or Tools: Code reviews are often facilitated using specialized tools or integrated development environments (IDEs) that allow reviewers to view the code changes and leave feedback.

4. Code Review Checklist: Reviewers may refer to a code review checklist or coding standards to ensure that the code follows established guidelines.

Code Review Process

NU MOA

5. Code Inspection: Reviewers examine the code thoroughly, looking for issues such as: syntax errors, logic errors, performance bottlenecks, security vulnerabilities, coding standard consistencies, adherence to best practices along with the code's overall design, scalability, and maintainability.

6. Feedbacks and Comments: Reviewers provide comments within the code review tool, explaining any issues they find. These comments should be clear, constructive, and specific, making it easier for the author to understand the feedback.

7. Discussion: The author and reviewers engage in a discussion about the code changes. This dialogue can involve clarifications, explanations, and suggestions for improvement.

9. Revisions: The author makes necessary changes based on the feedback received and continues to engage in discussions until all concerns are addressed. This may involve multiple review iterations.

Code Review Process

10. Approval: Once reviewers are satisfied with the code changes and all concerns have been resolved, they approve the code for integration.

11. Integration: The approved code changes are integrated into the main codebase, typically using version control systems.

12. Follow-Up: After integration, it's crucial to monitor the code in the production environment to ensure that the changes work as expected and do not introduce new issues.

13. Documentation and Closure: The code review process is documented for future reference, and the review is officially closed. This documentation may include the review comments, decisions made, and any action items.

Threat Modelling

Threat modeling is a structured approach to identifying and prioritizing potential threats to a system. The threat modeling process includes determining the value that potential mitigations would have in reducing or neutralizing these threats.

Threat modeling works to identify, communicate, and understand threats and mitigations within the context of protecting something of value.

Threat modeling also provides a clear ‘line of sight’ across a project that can be used to justify other security efforts.

Purpose of Threat Modeling

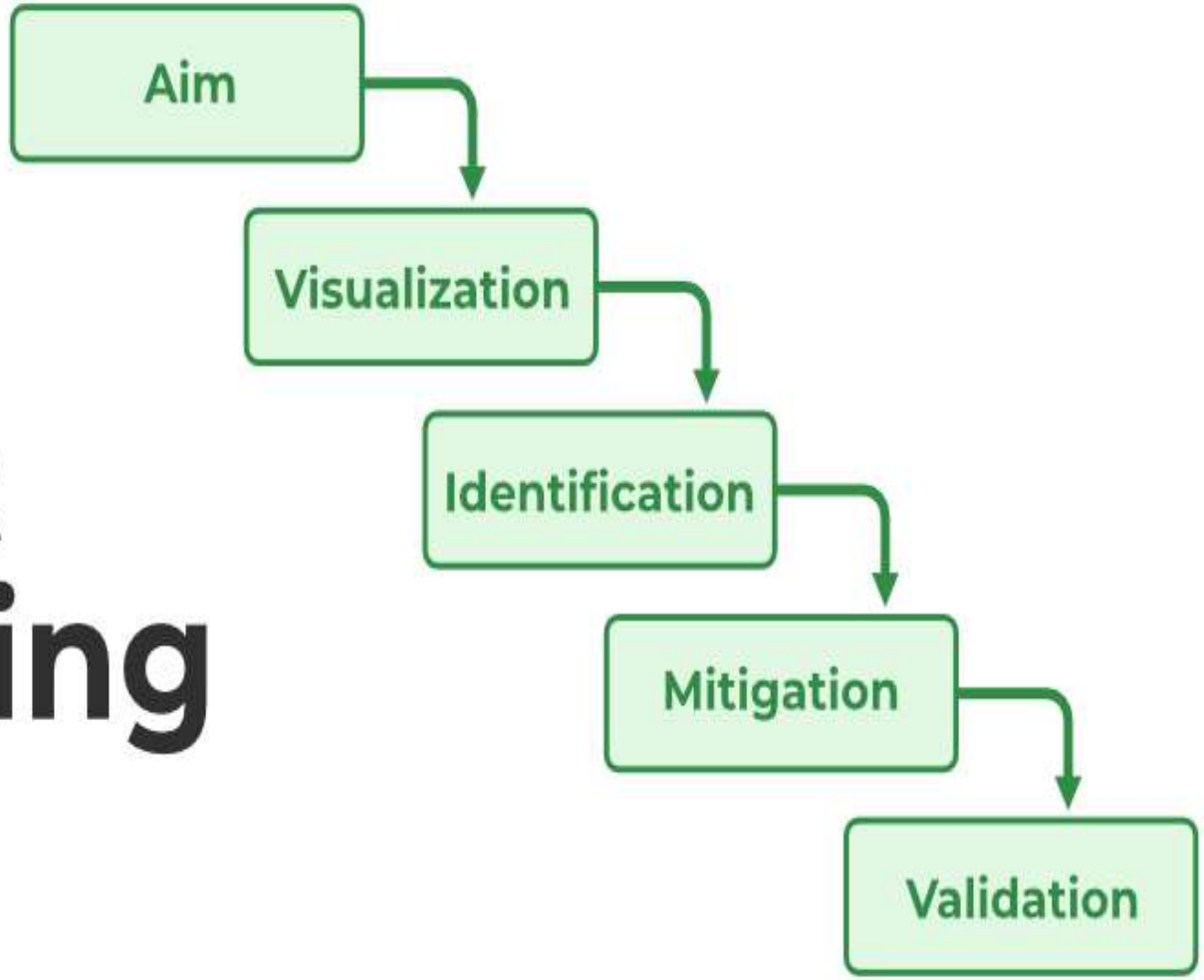
NU MOA

The purpose of Threat modeling is to identify, communicate, and understand threats and mitigation to the organization's stakeholders as early as possible.

Documentation from this process provides system analysts and defenders with a complete analysis of probable attackers' profiles, the most likely attack vectors, and the assets most desired by the attacker.

NU MOA

Threat Modeling Process



Threat Modeling Process

1. Aim

The target before approaching Threat Modeling must be clear within ourselves what we will achieve from Threat Modeling, that is our application must follow the CIA Triad.

- Confidentiality: It helps in protecting data from unauthorized access.
- Integrity: It helps in preventing restricted changes.
- Availability: It helps in performing important tasks under certain attacks.

Threat Modeling Process

2. Visualization

Here, we will deal with what we are going to build. We must have a document overview of the application which helps in making our process easier. Here we will build diagrams that will help us in making our process easier.

It can be done in two ways:

- Data Flow Diagram: It helps in showing how the flow of data occurs in the system.
- Process Flow Diagram: It helps in finding the process of the system that from where users interact in the system, and how the system works internally.

Threat Modeling Process

3. Threat Identification

Here we are going to deal with how we can identify threats or what can go wrong in the process.

By analyzing the images of the previous section, you have found how threats can be identified.

4. Mitigation

Here, we are going to deal with what we will do about the Threats. Here we will review the layers to identify the required vulnerabilities.

Mitigation involves a continuous investigation of each vulnerability so that the most effective efforts can be designed.

Threat Modeling Process

5. Validation

This is the final step in the process of Threat Modeling, here we are going to deal with whether we have done a good job or not.

Have all the threats been mitigated or not? We will check the changes and as threat modeling is not a one-time activity, we have to regularly watch these things.

Threat Modeling Methodology

- **Threat modeling methodology** refers to a structured approach or framework used to identify, assess, and prioritize potential security threats in a system or application.

Popular Threat Modeling Methodologies:

1. STRIDE:

STRIDE is a threat modeling methodology developed by Microsoft, focusing on the types of threats and how to categorize them. It’s especially useful for systems with complex interactions, helping teams think through potential attack vectors.

- **S**poofing: Impersonating something or someone.
- **T**ampering: Modifying data or system resources.
- **R**epudiation: Denying actions or transactions.
- **I**nformation Disclosure: Exposing confidential data to unauthorized parties.
- **D**enial of Service: Disrupting service or access to resources.
- **E**levation of Privilege: Gaining unauthorized access or privileges.

Popular Threat Modeling Methodologies:

2. PASTA (Process for Attack Simulation and Threat Analysis):

PASTA is a risk-centric methodology that aims to simulate potential attacks to assess the risk posed by different threats. It's designed to be a **high-level** approach to understanding the **attacker's perspective**.

PASTA consists of seven stages:

- Define the objectives of the analysis.
- Identify the attack surface.
- Identify threats and vulnerabilities.
- Perform risk analysis.
- Simulate attacks to assess the real-world impact.
- Define mitigation strategies.
- Reevaluate and improve the system.

Popular Threat Modeling Methodologies:

NU MOA

3. OCTAVE (Operationally Critical Threat, Asset, and Vulnerability Evaluation)

OCTAVE is a **strategic** threat modeling methodology that emphasizes the organizational risk management perspective.

Unlike other methodologies that focus primarily on technology, OCTAVE takes into account the people, processes, and assets involved in the system.

OCTAVE is divided into three phases:

Phase 1: Build Asset-Based Threat Profiles: Identify assets, security requirements, and business processes.

Phase 2: Identify and Analyze Threats: Identify threats and vulnerabilities to assets, including potential adversaries and attack scenarios.

Phase 3: Develop Security Strategy: Develop a risk-based security plan and mitigation strategy to protect critical assets.

Popular Threat Modeling Methodologies:

NU MOA

4. VAST (Visual, Agile, and Simple Threat modeling)

VAST was created to address the need for **visual** and **agile** threat modeling, especially in environments where systems are constantly evolving. It focuses on simplifying the process and integrating threat modeling into agile development practices.

VAST's main principles:

- **Visual Threat Modeling**: Creates visual models to identify potential threats. These diagrams often show data flows, components, and interactions in easy-to-understand formats.
- **Agility**: VAST integrates with agile software development practices, allowing for continuous assessment of risks as the system evolves.
- **Simplicity**: Aims to avoid overwhelming teams with unnecessary complexity, making threat modeling more accessible for developers, business stakeholders, and security teams.

Popular Threat Modeling Methodologies:

NU MOA

5. LINDDUN

LINDDUN is a methodology focused on **privacy threat modeling**, especially useful for systems handling personal data and meeting privacy regulations.

It is tailored for assessing **data privacy threats** rather than just security threats.

LINDDUN stands for:

- **L**inkability: The risk of linking data between different datasets or contexts.
- **I**dentity Disclosure: Exposing someone's identity without consent.
- **N**on-repudiation: Ensuring that actions or data cannot be denied (such as a user denying that they accessed specific data).
- **D**etectability: The ability of an attacker to detect the existence of sensitive information.
- **D**isclosure of Information: Unauthorized release of private information.
- **U**nawareness: When individuals are unaware of how their data is being used or processed.
- **N**on-compliance: Violating data protection laws and regulations (e.g., GDPR).

Popular Threat Modeling Methodologies:

NU MOA

6. Attack Tree Analysis

Attack Tree Analysis is a **hierarchical model** used to represent attacks on a system as a tree, with the goal of breaking down the attack into simpler sub-goals. Each node represents an attack step, and the leaves of the tree are basic attack methods or techniques.

Key components:

- **Root Node**: Represents the high-level goal or objective of the attack (e.g., gaining unauthorized access to a system).
- **Sub-Goals**: Nodes representing steps to achieve the root goal (e.g., exploiting a vulnerability, bypassing authentication).
- **Leaf Nodes**: Specific attack techniques or actions (e.g., using stolen credentials, exploiting a bug).

NU MOA

The choice of threat modeling methodology largely depends on the context of your system and objectives:

STRIDE is great for identifying general threats and is widely applicable to many kinds of systems.

PASTA is suited for organizations that want to take a **risk-centric** approach, especially for large, complex systems.

OCTAVE is ideal for organizations looking for **strategic risk management** with an emphasis on assets and organizational processes.

VAST is perfect for **agile development** environments and systems that need quick, iterative threat analysis.

LINDDUN is designed specifically for systems where **privacy concerns** are paramount, especially regarding data protection regulations.

Attack Tree Analysis helps in visualizing and breaking down potential attack scenarios in a methodical, **hierarchical** manner.